

```
pip install keras_tuner
```

[Show hidden output](#)

```
import pandas as pd
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from keras_tuner import RandomSearch
```

```
# Step 1: Load the dataset - Make sure the file is in the same folder
print("Dataset Loaded ")
data = pd.read_csv("/content/Crop_recommendation.csv")
print(data.head())
```

[Show hidden output](#)

```
# Step 2: Prepare Features (X) and Target (y)
X = data[['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall']]
y = data['label']
```

```
# Step 3: Encode the crop names into numbers
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)
```

```
# Step 4: Split into Train and Test sets
X_train, X_test, y_train, y_test = train_test_split(
X, y_encoded, test_size=0.2, random_state=42)
```

```
# Step 5: Scale the features (standardize values)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Step 6: Define a function to build the model with tunable parameters
def build_model(hp):
    model = Sequential()

    # First hidden layer with variable units
    model.add(
        Dense(
            units=hp.Int('units_1', 32, 256, step=32),
            activation='relu',
            input_shape=(X_train.shape[1],)
        )
    )

    # Second hidden layer with variable units
    model.add(
        Dense(
            units=hp.Int('units_2', 32, 256, step=32),
            activation='relu'
        )
    )

    # Output layer (22 crop classes)
    model.add(
        Dense(
            len(np.unique(y_encoded)),
            activation='softmax'
        )
    )

    # Compile the model
    model.compile(
```

```
optimizer=Adam(
    learning_rate=hp.Float('lr', 1e-4, 1e-2, sampling='LOG')
),
loss='sparse_categorical_crossentropy',
metrics=['accuracy']
)

return model
```

```
# Step 7: Setup Keras Tuner for Hyperparameter Tuning
tuner = RandomSearch(
    build_model,
    objective='val_accuracy',
    max_trials=5, # Number of combinations to try
    executions_per_trial=1, # Run each trial once
    directory='tuner_results',
    project_name='crop_recommendation',
)
```

[Show hidden output](#)

```
# Step 8: Add EarlyStopping to avoid overfitting
early_stop = EarlyStopping(monitor='val_loss', patience=3)
```

```
# Step 9: Start the tuning process
print("Starting Hyperparameter Tuning...")

tuner.search(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=32,
    callbacks=[early_stop],
    verbose=1
)
```

[Show hidden output](#)

```
# Step 10: Get the best model
best_model = tuner.get_best_models(1)[0]
```

[Show hidden output](#)

```
# Step 11: Train the best model again and store training history
history = best_model.fit(X_train, y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=32,
    callbacks=[early_stop],
    verbose=1)
```

[Show hidden output](#)

```
# Step 12: Evaluate the model
test_loss, test_acc = best_model.evaluate(X_test, y_test, verbose=1)
print(f"\nTest Accuracy: {test_acc * 100:.2f}%")
```

[Show hidden output](#)

```
# Step 13: Show best hyperparameters
best_hp = tuner.get_best_hyperparameters(1)[0]
print("\n Best Hyperparameters Found:")
print(" - Units in Layer 1:", best_hp.get('units_1'))
print(" - Units in Layer 2:", best_hp.get('units_2'))
print(" - Learning Rate:", best_hp.get('lr'))
```

[Show hidden output](#)

```
# Step 14: Plot Accuracy and Loss
plt.figure(figsize=(12, 5))
# Accuracy plot
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy', marker='o')
plt.plot(history.history['val_accuracy'], label='Val Accuracy', marker='o')
plt.title("Accuracy vs Epochs",loc='left')
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.text(1.0,1.02,'(2311cs010480)-N.Rishik',transform=plt.gca().transAxes,ha='right')
plt.legend()
plt.grid(True)
```

[Show hidden output](#)

```
# Loss plot
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss', marker='o')
plt.plot(history.history['val_loss'], label='Val Loss', marker='o')
plt.title("Loss vs Epochs",loc='left')
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.text(1.0,1.02,'(2311cs010480)-N.Rishik',transform=plt.gca().transAxes,ha='right')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

[Show hidden output](#)

```
# Step 15: Make a prediction on a sample
sample = X_test[5:6]

predicted_crop = best_model.predict(sample)

predicted_label = label_encoder.inverse_transform(
    [np.argmax(predicted_crop)]
)

print(f"\nPredicted Crop for Sample: {predicted_label[0]}")
```

[Show hidden output](#)